

Recurrent Gap-Time Survival, Rank Tests, and Competing Risks Analysis: `recurSurvTests`

John Muschelli

2026-09-23

Abstract

Nonparametric recurrent-event survival curves are available in the statistical literature, but openly available implementations of tests for differences between those curves have been limited. `recurSurvTests` provides R implementations for estimating and comparing recurrent curves on their intended scales: (i) Wang–Chang marginal recurrent gap-time survival and Luo–Huang weighted-risk-set rank tests, (ii) the Pena–Strawderman–Hollander generalized product-limit estimator, (iii) a separately documented Zhao et al. extended-rank implementation with a sensitivity variance path, and (iv) Sivadasan–Sankaran recurrent cause-specific cumulative-incidence functions and their Wald-type comparison. The package is designed to enable tests whose risk sets, weights, and null hypotheses correspond to the curve an investigator has chosen to estimate. Because recurrent gap-time survival, calendar-time event burden, and recurrent competing-risk incidence are distinct estimands, the software makes those choices explicit rather than treating their tests as substitutes. This manuscript documents the methods, data conventions, numerical checks against prior R implementations where available, and reproducible simulation designs for published-style and polysomnography-shaped settings. Available simulation cells are reported with Monte Carlo uncertainty and explicit missing-cell indicators; the manuscript regenerates its comparisons when additional completed runs are supplied.

Keywords: recurrent events, gap time, Wang–Chang estimator, weighted risk set, product-

limit estimator, competing risks, R

Introduction

Recurrent events occur when a unit can experience an event repeatedly during observation: hospital admissions, equipment failures, sleep-stage bouts, and recurrent infections are examples. A frequent implementation problem is that a method described as a “recurrent survival curve” may not estimate the same quantity as another method with a similar name. This distinction is especially important when the number of observed recurrences is associated with a participant’s propensity for short gaps.

This distinction creates a practical inferential problem, not merely a naming problem. An investigator may plot a recurrent gap-time survival curve and want to test whether that curve differs by treatment, diagnosis, or visit. A test for a calendar-time mean cumulative function, a regression coefficient, or a first-event survival curve instead answers a different question. Thus, the estimand used for the curve and the estimand targeted by its comparison must be aligned before a reported p-value has its intended interpretation.

Substantial software already supports recurrent-event regression. For example, the Andersen–Gill [1], PWP [2], and related Cox-model formulations provide score, Wald, and likelihood-based inference for regression coefficients; mixed-effects and frailty extensions are available in `coxme` [3] and `frailtypack` [4]. These are important tools when a conditional covariate or frailty effect is the scientific target. They do not, however, supply a nonparametric global test for equality of an estimated recurrent gap-time survival curve. Existing recurrent-curve R software has also made important contributions: `survrec` [5] includes Wang–Chang and Pena–Strawderman–Hollander estimators, and `newTestSurvRec` [6] includes recurrent-event curve estimators and rank-test procedures. Their available estimators and testing paths do not all target the same curve, so a test cannot be transferred between them solely on the basis of a shared label such as “recurrent survival.”

`recurSurvTests` fills the resulting open-source implementation gap by enabling tests that correspond to the recurrent curves under analysis. It is not a general recurrent-event modeling framework. Its primary aim is to let users estimate a clearly defined recurrent curve and

conduct a comparison whose risk set, weighting, and null hypothesis match that curve—in particular, the Wang–Chang marginal gap-time survival curve and its Luo–Huang weighted-risk- set comparison. The package also keeps alternative estimands explicit, rather than presenting their tests as interchangeable.

Three scientific targets are kept separate throughout:

1. Marginal recurrent **gap-time survival**, the target of Wang–Chang and the Luo–Huang rank test.
2. Mean cumulative recurrent-event burden on the **calendar-time** scale, which is not implemented here.
3. Recurrent cause-specific cumulative incidence for **gap times**, the target of the Sivadasan–Sankaran estimators.

This separation avoids calling all recurrent-event comparisons “log-rank tests.” In particular, the METRON recurrent-CIF procedure is a weighted contrast/Wald-type procedure and is not a log-rank test.

Data convention and software design

All gap-time survival functions accept a long data frame with one row per observed gap. For subject i , rows are ordered by episode. Completed gaps have $\delta_{ij} = 1$; the final row is the terminal right-censored gap with $\delta_{ij} = 0$. The observed gap duration is Y_{ij} . An optional **episode** column makes the ordering explicit. This convention permits a participant with no completed recurrence to contribute a single terminal censored gap.

```
library(recurSurvTests)

gaps <- data.frame(
  id = rep(1:4, each = 3),
  episode = rep(1:3, 4),
  gap = c(2, 3, 4, 1, 4, 3, 2, 2, 5, 1, 3, 4),
  status = rep(c(1, 1, 0), 4),
  arm = rep(c("control", "treatment"), each = 6),
```

```

cause = c("A", "B", NA, "A", "A", NA, "B", "A", NA, "B", "B", NA)
)

```

Internal validation enforces this final-censoring convention, nonnegative gap durations, and subject-level constancy of group and paired-visit labels. The core helpers turn long data into subject records and form risk/event components at distinct completed gap times. Keeping these components visible is intended to make formula-to-code comparison possible.

Recurrent gap-time survival estimators

Wang–Chang marginal survival

Let $m_i^* = 1$ for a participant with no complete recurrence and otherwise let m_i^* equal the number of completed gaps. The Wang–Chang weighted risk and event increments [7, 8] are

$$R_{WC}(t) = \sum_i \frac{1}{m_i^*} \sum_{j=1}^{m_i^*} I(Y_{ij} \geq t), \quad dN_{WC}(t) = \sum_i \frac{1}{m_i^*} \sum_{j=1}^{m_i^*} I(Y_{ij} = t, \delta_{ij} = 1).$$

`wc_surv()` calculates

$$\widehat{S}_{WC}(t) = \prod_{u \leq t} \left\{ 1 - \frac{dN_{WC}(u)}{R_{WC}(u)} \right\}.$$

The inverse- m_i^* weighting prevents participants with many observed events from receiving proportionally more influence on the marginal gap distribution. The implementation uses the weighted-risk-set representation shown by Luo and Huang [8] to be algebraically equivalent to the estimator of Wang and Chang [7].

PSH and pooled Kaplan–Meier

The Pena–Strawderman–Hollander (PSH) generalized product-limit estimator [9] uses all observed gap records with unit weight:

$$R_{PSH}(t) = \sum_{i,j} I(Y_{ij} \geq t), \quad dN_{PSH}(t) = \sum_{i,j} I(Y_{ij} = t, \delta_{ij} = 1),$$

$$\widehat{S}_{PSH}(t) = \prod_{u \leq t} \left\{ 1 - \frac{dN_{PSH}(u)}{R_{PSH}(u)} \right\}.$$

Therefore `psh_surv()` is numerically identical to an ordinary pooled Kaplan–Meier calculation on the same long gap records, including terminal censored gaps. They do not diverge merely because censoring is increased. They can appear to differ only if “KM” refers to a different data construction, for example first gaps only, a calendar-time analysis, different treatment of terminal censoring, or weights. PSH has a renewal/IID gap-model interpretation; that interpretation is distinct from the Wang–Chang marginal estimand.

The legacy PSH implementations return an independence-style no-ties Greenwood/Nelson–Aalen approximation, $\widehat{S}_{PSH}(t)^2 \sum_{u \leq t} dN(u)/R(u)^2$. It is not generally appropriate when within-subject gaps are dependent or event counts are informative. In such settings, clustered uncertainty methods are needed; a point estimator alone does not solve the inference problem.

Subject-bootstrap confidence intervals

`recurrent_surv_ci()` resamples complete participant histories and refits either curve. This is required because recurrent gaps from one participant are dependent, and because the WC weights are functions of the complete history. The default percentile intervals are pointwise: at a pre-specified time t , they target $P\{S(t) \in CI(t)\} = 1 - \alpha$. They are the conventional uncertainty summary for a plotted curve, but do not support a joint claim about every point of the curve. For that purpose, `interval = "simultaneous"` uses a subject-bootstrap max- t critical value to form a band over an explicitly supplied finite time grid. The associated coverage simulation estimates pointwise and joint coverage separately; its results remain pending until the configured run has completed.

The default number of bootstrap draws is $B = 999$, consistent with common bootstrap and permutation practice. It is adequate for routine exploration, but a two-sided 95% percentile interval estimates each 2.5% tail from only about 25 draws. For final 95% percentile limits or max- t critical values, the documentation recommends $B \geq 2000$ when computation permits, yielding about 50 draws in each percentile tail and reducing Monte-Carlo variation.

The coverage statement must match the reported quantity. A pointwise 95% interval has ap-

114 proximately 95% coverage for $S(t_0)$ at one pre-specified gap time t_0 . Drawing such intervals
 115 at multiple times does not yield 95% joint coverage of the full curve. The simultaneous max- t
 116 band instead targets $P\{S(t) \text{ is covered for every } t \in \mathcal{T}\} \approx 0.95$ for the user-supplied finite
 117 grid $\mathcal{T}$; it is correspondingly wider. Neither output is a confidence interval for a survival quan-
 118 tile (an inverse-curve parameter such as a median gap) or for a between-group curve differ-
 119 ence. Those are separate targets. `recurrent_surv_quantile_ci()` provides a whole-subject
 120 bootstrap percentile interval for a survival quantile; it does not invert the independence-style
 121 normal curve limits because that inversion does not account for within-subject clustering.
 122 A between-group quantile or curve-difference interval requires its own two-group cluster-
 123 resampling contrast. `recurrent_surv_contrast_ci()` now supplies the latter for pointwise
 124 differences, ratios, or log-ratios and for a finite-grid simultaneous band, resampling within
 125 independent arms or complete paired-visit clusters. `recurrent_surv_rmst_ci()` supplies a
 126 clustered bootstrap interval for the restricted mean gap time, which remains defined when
 127 the median is not reached.

128 This interface complements, rather than replaces, prior software. The `survrec` WC and PSH
 129 fitters return pointwise standard errors, and its plotting method draws normal-approximation
 130 pointwise limits. `survrec::survdiff()` also performs WC or PSH bootstrap resampling
 131 for a *selected survival quantile*, such as a median, and returns a `boot` object for subsequent
 132 interval calculation. Thus bootstrap inference was not absent from earlier R work. The
 133 standard errors returned by the inspected `newTestSurvRec` WC and PSH fitters are the
 134 independence-style Nelson–Aalen/Greenwood approximation,

$$\widehat{SE}\{\widehat{S}(t)\} = \widehat{S}(t) \left\{ \sum_{u \leq t} \frac{dN(u)}{R(u)^2} \right\}^{1/2}.$$

135 Consequently, those values can be used to form a clipped normal pointwise curve interval,
 136 $\widehat{S}(t) \pm z_{1-\alpha/2} \widehat{SE}(t)$; they are neither bootstrap intervals nor simultaneous bands. The same
 137 independence-style form, evaluated with this package's own event and risk increments, is
 138 available with `ci_method = "normal"` to make the legacy-style capability explicit. It repro-
 139 duces the PSH standard-error calculation on the same long records; for WC it is an analogous
 140 approximation for the Wang–Chang-equivalent weighted risk set, not a claim of numerical

141 replication of another package's variance code. It should not be relied on for dependent
142 recurrent gaps or informative event counts, because the displayed variance does not account
143 for subject-level clustering. In contrast, `ci_method = "bootstrap"` provides a whole-curve
144 subject-bootstrap workflow: percentile intervals for fixed times or a max- t band for a fi-
145 nite grid. The resampling itself is conceptually straightforward; its value here is a uniform
146 analyst-facing tool that makes the resampling unit, estimator, time grid, and coverage claim
147 explicit and consistent across WC and PSH analyses.

148 Comparing curves

149 The following executable example compares Wang–Chang, PSH, and an independently com-
150 puted pooled Kaplan–Meier curve from `survival` [10] on the same records. It illustrates the
151 estimands, rather than reproducing a figure from a source paper.

```
wc <- wc_surv(gaps, episode = "episode")
psh <- psh_surv(gaps, episode = "episode")

step_at <- function(fit, grid) vapply(grid, function(t) {
  ii <- which(fit$time <= t)
  if (length(ii)) fit$surv[max(ii)] else 1
}, numeric(1))

grid <- sort(unique(c(0, wc$time, psh$time)))
km <- survival::survfit(survival::Surv(gap, status) ~ 1, data = gaps)
km_curve <- data.frame(time = km$time, surv = km$surv)
stopifnot(max(abs(step_at(psh, grid) - step_at(km_curve, grid))) < 1e-12)
par(mfrow = c(1, 2))
plot(grid, step_at(wc, grid), type = "s", ylim = c(0, 1),
     xlab = "Gap time", ylab = "Survival")
lines(grid, step_at(psh, grid), type = "s", col = "firebrick", lty = 2)
legend("topright", c("Wang-Chang", "PSH / pooled KM"),
     col = c("black", "firebrick"), lty = 1:2, bty = "n")
plot(grid, step_at(wc, grid) - step_at(psh, grid), type = "s",
```

```

xlab = "Gap time", ylab = "Wang--Chang minus PSH/KM")
abline(h = 0, lty = 2)

```

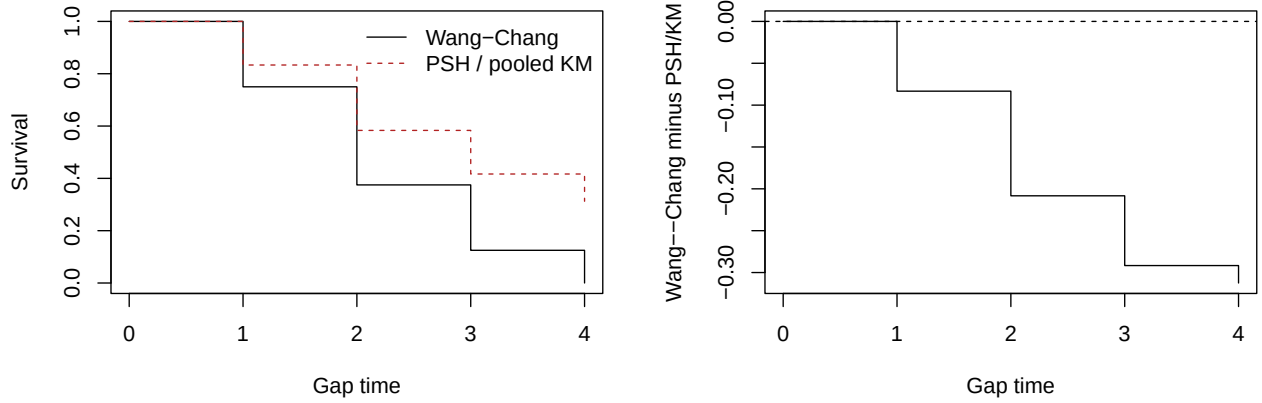


Figure 1: Wang–Chang and PSH/pooled Kaplan–Meier survival on the example records, and their difference.

Rank tests and paired visits

`wc_logrank()` implements the two-group Luo–Huang weighted-risk-set [8] G_ρ^* statistic. At each event time it contrasts weighted observed events with their group-specific weighted risk-set expectation. With $\rho = 0$, it is the recurrent gap-time log-rank analogue; $\rho = 1$ gives a Peto–Prentice-type weight. The variance is based on subject-level martingale-style contributions to retain within-subject gap dependence.

`zhao_rank_test()` is separate. It implements the extended LR, Gehan–Breslow, and Peto–Prentice score construction described by Zhao et al. [11]. Its default `pooled_risk` residual is assessed by the simulations below; current results do not establish published-table reproduction. The group-risk denominator printed in Zhao et al.’s Equation 6 is retained as the explicit `zhao_eq6` sensitivity/reproduction path. Neither function should be described as an exact substitute for the other.

For two paired visits per participant, `wc_pair permutation()` swaps labels within participant pairs and recalculates G_ρ^* . Its p-value tests the within-pair label-exchangeability null. The independent-arm analytical p-value from `wc_logrank()` is returned for context but is not a paired analysis.

Recurrent competing risks

`ss_rcif()` implements recurrent cause-specific cumulative-incidence functions following Sivadasan and Sankaran [12]. The default shared weighting keeps pooled and cause-specific increments additive. `cause_weighting = "literal_eq15"` is retained as a sensitivity/reproduction option because literal use of the printed Equation 15 denominator can yield nonadditive increments.

`ss_rcif_equal_causes_test()` is a subject-bootstrap Wald reconstruction of the METRON equal-cause test [13]. It compares integrated RCIF contrasts and must not be called a log-rank test.

Validation against prior implementations

The following checks rerun the continuous-data examples from the reference vignettes with fixed seeds. Each row compares the same event-time grid and survival values with the indicated installed package [5, 6]. Optional packages are not downloaded during rendering; missing installations produce an explicit unavailable result. The package versions and discrepancies are computed at render time, not copied from previous output.

```
reference_checks <- manuscript_reference_checks()
reference_display <- reference_checks
reference_display$Max_absolute_difference <- ifelse(
  is.na(reference_checks$Max_absolute_difference), "Not run",
  formatC(reference_checks$Max_absolute_difference, format = "e", digits = 3))
knitr::kable(reference_display)
```

Table 1: Numerical comparisons for Wang–Chang and PSH estimators. Missing differences indicate an unavailable reference package.

Scenario	Estimator	Reference	Version	Max_absolute_difference
100 subjects, 1-5 events	WC	survrec	1.2.5	1.055e-15
100 subjects, 1-5 events	WC	newTestSurvRec	1.0.2	3.333e-09
100 subjects, 1-5 events	PSH	survrec	1.2.5	0.000e+00

Table 1: Numerical comparisons for Wang–Chang and PSH estimators. Missing differences indicate an unavailable reference package.

Scenario	Estimator	Reference	Version	Max_absolute_difference
100 subjects, 1-5 events	PSH	newTestSurvRec	1.0.2	0.000e+00
80 subjects, 0-6 events	WC	survrec	1.2.5	5.551e-17
80 subjects, 0-6 events	WC	newTestSurvRec	1.0.2	4.999e-09
80 subjects, 0-6 events	PSH	survrec	1.2.5	0.000e+00
80 subjects, 0-6 events	PSH	newTestSurvRec	1.0.2	0.000e+00

These are software comparisons for the estimators of Wang and Chang [7] and Peña et al. [9]. They are not reproductions of the simulation tables in those papers. In particular, newTestSurvRec’s LRrec pathway is associated with PSH-style estimation and is not a numerical reference for the Luo–Huang rank test.

```
rank_example <- do.call(rbind, lapply(c(0, 1), function(rho) {
  fit <- wc_logrank(gaps, group = "arm", episode = "episode", rho = rho)
  data.frame(Method = paste0("Luo-Huang, rho = ", rho),
             z = fit$z, p_value = fit$p.value)
}))
knitr::kable(rank_example, digits = 4)
```

Table 2: Executable rank-test example on the four-subject illustration. These values demonstrate the interface and are not simulation validation.

Method	z	p_value
Luo-Huang, rho = 0	1.9612	0.0499
Luo-Huang, rho = 1	1.7598	0.0784

Published-benchmark placeholder for Wang–Chang and Luo–Huang. The curve in Figure 1, the independent pooled-KM check, and Table 1 provide reproducible estima-

tor checks. Direct numerical reproduction of the original Wang–Chang and Luo–Huang simulation tables [7, 8] remains a separate validation task requiring a matched design and transcribed published benchmarks. Zhao’s results below assess a different rank-test construction and cannot fill that role. The existing K-group PSG simulation is an application-specific extension check, not a reproduction of Luo–Huang’s two-group study.

Simulation validation

Zhao published-table comparison

This section compares completed runs with the **New method** columns of Tables 1–3 in Zhao et al. [11]. Table 1 reports the SD of the standardized null statistic; Table 2 reports one-sided type-I error at $\alpha = 0.025$; Table 3 reports power. LR, PP, and GB denote log-rank, Peto–Prentice, and Gehan–Breslow weights. OLR and Jung–Jeong columns are not used as benchmarks for this implementation.

`data-raw/zhao-2020-table-simulations.R` defines 120 cells with four baseline distributions, three heterogeneity ranges, 100 subjects per arm, study end 180 days, and entry uniform on $(0, 180)$. Subject gap multipliers follow $Z \sim U(0.5, 1.5)$, $U(0.1, 1.9)$, or $U(0.01, 1.99)$, labeled low, medium, and high heterogeneity. The intended numbers of simulated datasets are 100,000 per null cell and 10,000 per power cell. Treatment gaps in Table 3 are multiplied by $\exp(0.25)$ or $\exp(0.5)$; for Weibull shape 2 these correspond to changes of 0.5 and 1 in $-\log(\lambda)$.

The baseline laws in the corrected generator match Table 1: exponential rate $\exp(-4)$, Weibull shape 2 and scale $\exp(4)$, log-normal parameters $(\mu, \sigma) = (4, 0.5)$, and log-logistic survival $S(t) = \{1 + [\exp(-4)t]^2\}^{-1}$. The local generator uses administrative censoring. Zhao used `survsim::rec.ev.surv()` [14] and discussed an additional noninformative censoring mechanism without full parameter details in Section 3. Thus matching baseline distributions does not establish identical simulation designs.

The reference CSV, `data-raw/zhao-2020-published-tables.csv`, was transcribed from pages 93–95 of the publisher PDF. **Table 2 has a scale ambiguity:** its caption says “(*100)” but prints integers such as 27 and 25. We interpret these as 0.027 and 0.025,

217 consistent with Section 3.2’s nominal 0.025 level and interpretation. This is an explicit,
 218 unconfirmed correction; both raw printed values and interpreted probabilities are retained.
 219 Table 2 comparisons are conditional on it.

```
zhao <- manuscript_zhao(repo_root)
zhao_status <- manuscript_zhao_status(zhao)
zhao_available <- sum(zhao$available)
zhao_comparable <- sum(zhao$available & zhao$design_eligible)
zhao_discrepant <- sum(zhao$comparison == "discrepant")
zhao_method <- if (zhao_available) unique(zhao$variance_method) else "not available"
```

220 At this render, **86 of 120 cells** are available, of which 86 satisfy the recorded design-
 221 eligibility checks. The variance convention recorded in these files is **pooled_risk**. Avail-
 222 ability is assessed from files, not scheduler status. Missing files are shown as *Pending*; they
 223 do not contribute zero rejection rates or enter comparison summaries. **pooled_risk** and
 224 the printed Equation 6 group-risk convention (**zhao_eq6**) remain distinct; the importer re-
 225 jects mixtures of variance conventions and inconsistent settings. The **zhao_eq6** comparison
 226 remains pending unless a separate, consistently configured result directory is supplied.

```
knitr::kable(zhao_status)
```

Table 3: Available and missing cells in the Zhao validation design. Discrepant counts use the exploratory Monte Carlo criterion below.

Table	Planned	Available	Comparable	Missing	Excluded	Replicates	_per_available	Discrepant
1	12	9	9	3	0	100000		3
2	36	24	24	12	0	100000		4
3	72	53	53	19	0	10000		53

227 An earlier log-logistic generator used an incorrect scale and inverse transform, producing
 228 vastly too many recurrences. Those outputs cannot validate Zhao’s design. Corrected runs
 229 carry **generator_version = "zhao_table1_loglogistic_v2"**; legacy log-logistic outputs
 230 are displayed as *Excluded*. Existing results for the other three distributions are retained.

231 Rerendering automatically incorporates newly completed, eligible cells, including all 120
 232 when available.

233 Monte Carlo uncertainty and current findings

234 For a rejection proportion $\hat{p}$ based on B replicates, the Monte Carlo standard error is $\{\hat{p}(1 -$
 235 $\hat{p})/B\}^{1/2}$. Local rejection-rate intervals use Wilson’s formula. For empirical SDs in Table
 236 1 we use the normal-theory approximation $\text{MCSE}(s) = s/\sqrt{2(B-1)}$; replicate-level fourth
 237 moments are unavailable, so this is approximate. A cell receives an asterisk when

$$|\hat{\theta}_{\text{local}} - \hat{\theta}_{\text{paper}}| > 1.96\sqrt{\text{MCSE}_{\text{local}}^2 + \text{MCSE}_{\text{paper}}^2} + 0.0005.$$

238 The last term allows half a unit of rounding in the published tables. The calculation assumes
 239 independent simulation runs and uses the paper’s replicate counts. It is an exploratory cell-
 240 wise discrepancy screen, not an equivalence test or a multiplicity-adjusted global validation
 241 rule. A cell without an asterisk is not proven equivalent. Nominal calibration is assessed
 242 separately in the exported CSV by exact binomial tests against 0.025, with Holm adjustment
 243 across the 36 planned Table 2 cells.

244 Among the 86 comparable completed cells, **60 are discrepant** under the criterion above.

245 Across 53 available power cells, local minus published power ranges from **+0.46 to +33.44**
 246 **percentage points**. These results require investigation of the generator/censoring design
 247 and variance convention before claiming published-table reproduction.

248 The original RDS files retain summary moments, not individual simulated statistics or failed-
 249 p-value counts. Consequently, the null histogram in Zhao’s Figure 1 cannot be reconstructed
 250 from this batch. No histogram is inferred from a mean and SD. The figure below instead com-
 251 pares the reported quantities directly, while the following three tables retain the distribution,
 252 heterogeneity, alternative, and LR/PP/GB arrangement of Zhao’s tables.

`manuscript_zhao_plot(zhao)`

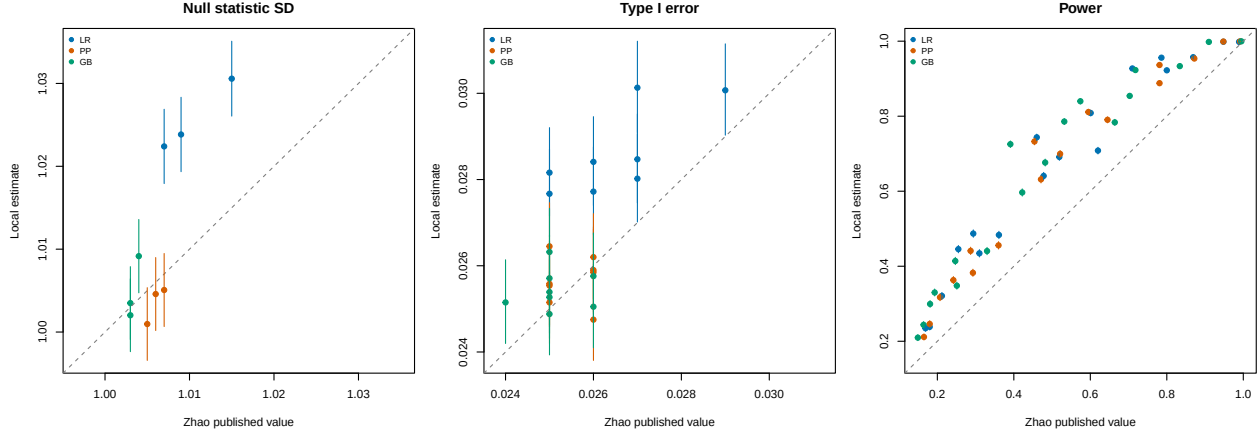


Figure 2: Published versus local Zhao simulation summaries. The dashed line is equality. Bars show local 95% Monte Carlo intervals (normal approximation for SDs, Wilson intervals for rejection rates). Paper uncertainty and rounding also enter the discrepancy screen. Only available, design-eligible cells are plotted.

```
knitr::kable(manuscript_zhao_table(zhao, "table1"))
```

Table 4: Comparison with Zhao Table 1: empirical SD of the standardized null statistic at medium heterogeneity. Parentheses contain local Monte Carlo SEs; an asterisk marks a discrepancy.

	LR	LR local	PP	PP local	GB	GB local
Distribution	Zhao	(MCSE)	Zhao	(MCSE)	Zhao	(MCSE)
exponential	1.007	1.0224 (0.0023)	1.005	1.0010	1.004	1.0092
		*		(0.0022)		(0.0023)
weibull	1.015	1.0306 (0.0023)	1.007	1.0051	1.003	1.0035
		*		(0.0022)		(0.0022)
lognormal	1.009	1.0238 (0.0023)	1.006	1.0046	1.003	1.0020
		*		(0.0022)		(0.0022)
loglogistic	1.009	Pending	0.997	Pending	0.998	Pending

```
knitr::kable(manuscript_zhao_table(zhao, "table2"))
```

Table 5: Comparison with Zhao Table 2: one-sided type-I error probabilities. Published integers are divided by 1000 as explained in the text. Parentheses contain local Monte Carlo SEs; an asterisk marks a discrepancy.

		LR	LR local	PP	PP local	GB	GB local
Heterogeneity	Distribution	Zhao	(MCSE)	Zhao	(MCSE)	Zhao	(MCSE)
low	exponential	0.027	0.0280 (0.0005)	0.026	0.0262 (0.0005)	0.026	Pending
low	weibull	0.029	0.0301 (0.0005)	0.026	0.0259 (0.0005)	0.025	0.0254 (0.0005)
low	lognormal	0.027	0.0301 (0.0005) *	0.025	0.0265 (0.0005)	0.026	0.0250 (0.0005)
low	loglogistic	0.026	Pending	0.026	Pending	0.025	Pending
medium	exponential	0.026	0.0284 (0.0005) *	0.025	0.0255 (0.0005)	0.026	0.0258 (0.0005)
medium	weibull	0.027	0.0285 (0.0005)	0.026	0.0259 (0.0005)	0.025	0.0263 (0.0005)
medium	lognormal	0.025	0.0277 (0.0005) *	0.025	0.0251 (0.0005)	0.025	0.0257 (0.0005)
medium	loglogistic	0.026	Pending	0.025	Pending	0.025	Pending
high	exponential	0.026	Pending	0.026	Pending	0.025	0.0249 (0.0005)
high	weibull	0.025	0.0282 (0.0005) *	0.025	0.0256 (0.0005)	0.024	0.0251 (0.0005)
high	lognormal	0.026	0.0277 (0.0005)	0.026	0.0248 (0.0005)	0.025	0.0253 (0.0005)
high	loglogistic	0.025	Pending	0.025	Pending	0.024	Pending

```
knitr::kable(manuscript_zhao_table(zhao, "table3"))
```

Table 6: Comparison with Zhao Table 3: one-sided power probabilities. Parentheses contain local Monte Carlo SEs; an asterisk marks a discrepancy. The log-time shift is 0.25 or 0.5 for all four distributions.

Heterogeneity	Distribution	Log-time shift	LR	LR local	PP	PP local	GB	GB local
			Zhao	(MCSE)	Zhao	(MCSE)	Zhao	(MCSE)
low	exponential	0.25	0.212	0.3209	0.207	0.3172	0.181	0.2994
				(0.0047) *				(0.0046) *
low	exponential	0.50	0.601	0.8085	0.595	0.8111	0.532	0.7858
				(0.0039) *				(0.0041) *
low	weibull	0.25	0.460	0.7437	0.454	0.7326	0.391	0.7254
				(0.0044) *				(0.0045) *
low	weibull	0.50	0.948	0.9985	0.948	0.9990	0.910	0.9980
				(0.0004) *				(0.0004) *
low	lognormal	0.25	0.620	0.7083	0.645	0.7904	0.664	0.7836
				(0.0045) *				(0.0041) *
low	lognormal	0.50	0.989	0.9982	0.992	0.9996	0.995	0.9996
				(0.0004) *				(0.0002) *
low	loglogistic	0.25	0.325	Pending	0.334	Pending	0.334	Pending
low	loglogistic	0.50	0.828	Pending	0.846	Pending	0.852	Pending
medium	exponential	0.25	0.180	0.2384	0.180	0.2462	0.164	0.2438
				(0.0043) *				(0.0043) *
medium	exponential	0.50	0.519	0.6914	0.521	0.6996	0.482	0.6765
				(0.0046) *				(0.0047) *
medium	weibull	0.25	0.294	0.4871	0.287	0.4410	0.247	0.4140
				(0.0050) *				(0.0049) *
medium	weibull	0.50	0.786	0.9560	0.781	0.9365	0.718	0.9230
				(0.0021) *				(0.0027) *
medium	lognormal	0.25	0.361	0.4834	0.360	0.4557	0.330	0.4405
				(0.0050) *				(0.0050) *

Table 6: Comparison with Zhao Table 3: one-sided power probabilities. Parentheses contain local Monte Carlo SEs; an asterisk marks a discrepancy. The log-time shift is 0.25 or 0.5 for all four distributions.

		Log-time	LR	LR local	PP	PP local	GB	GB local
Heterogeneity	Distribution	shift	Zhao	(MCSE)	Zhao	(MCSE)	Zhao	(MCSE)
medium	lognormal	0.50	0.869	0.9573 (0.0020) *	0.872	0.9537 (0.0021) *	0.834	0.9335 (0.0025) *
medium	loglogistic	0.25	0.242	Pending	0.247	Pending	0.230	Pending
medium	loglogistic	0.50	0.695	Pending	0.711	Pending	0.688	Pending
high	exponential	0.25	0.169	0.2347 (0.0042) *	0.165	0.2114 (0.0041) *	0.149	0.2096 (0.0041) *
high	exponential	0.50	0.478	0.6408 (0.0048) *	0.471	0.6316 (0.0048) *	0.422	0.5967 (0.0049) *
high	weibull	0.25	0.255	0.4456 (0.0050) *	0.242	0.3630 (0.0048) *	0.193	0.3300 (0.0047) *
high	weibull	0.50	0.710	0.9273 (0.0026) *	0.683	Pending	0.574	0.8399 (0.0037) *
high	lognormal	0.25	0.310	0.4345 (0.0050) *	0.293	0.3825 (0.0049) *	0.251	0.3481 (0.0048) *
high	lognormal	0.50	0.800	0.9223 (0.0027) *	0.781	0.8880 (0.0032) *	0.703	0.8541 (0.0035) *
high	loglogistic	0.25	0.210	Pending	0.212	Pending	0.191	Pending
high	loglogistic	0.50	0.614	Pending	0.617	Pending	0.569	Pending

Full-run placeholder. These same chunks will produce the full-data tables and plot after the remaining RDS files are copied into the input directory. Rendering never starts the 100,000-replicate jobs. Use a separate directory and `ZHAO_RESULTS_DIR` to render a different variance convention rather than pooling results across methods. The completed-cell findings above remain provisional until the design differences and observed discrepancies are resolved.

PSG-shaped simulation study

The PSG supplement script simulates recurrent bouts with: (i) subject frailty, (ii) visit effects, (iii) persistent burst/recovery states, (iv) 360–480 minute recordings with terminal censoring, (v) unequal independent-arm sizes (30 control, 45 treatment), and (vi) paired visits with shared participant frailty. Under the null, treatment and control gaps have the same distribution. Power alternatives multiply treatment gaps by 1.25 (longer bouts) or 0.80 (shorter bouts). The script evaluates G_0^* , G_1^* , Zhao pooled-risk sensitivity analyses, and paired label permutations where appropriate.

```
psg_file <- Sys.getenv("PSG_RESULTS_FILE", file.path(repo_root, "data-raw/psg-validation-
if (!grepl("^(/[A-Za-z]:)", psg_file)) psg_file <- file.path(repo_root, psg_file)
if (file.exists(psg_file)) {
  psg_results <- readRDS(psg_file)
  stopifnot(is.data.frame(psg_results),
    all(c("design", "alternative", "method", "n_sim", "B", "rejection_rate", "monte_carl
  cat("Available PSG summaries are shown with their actual replicate and permutation cou
  print(knitr::kable(psg_results, digits = 4))
} else {
  cat("**Results placeholder.** No combined PSG result file is available. The planned 1,
}
```

Results placeholder. No combined PSG result file is available. The planned 1,000-replicate/999-permutation results will be imported on rerender when `data-raw/psg-validation-results` is supplied. Set `PSG_RESULTS_FILE` to select a different result file.

K-group application-specific pilot

The following table imports the existing K-group output, if available. Its replicate count is displayed explicitly. It assesses the package’s extension of the weighted-risk-set score to three unequal arms under the PSG-shaped design in `data-raw/wc-logrank-k-validation-simulation.R`. It is not a published numerical benchmark from Luo and Huang [8], and a small pilot does not establish type-I-error control.

```

k_file <- file.path(repo_root, "data-raw/wc-logrank-k-validation-results.csv")
if (file.exists(k_file)) {
  k_results <- read.csv(k_file)
  stopifnot(all(c("scenario", "rejection_rate", "mc_se", "n_sim") %in% names(k_results)))
  print(knitr::kable(k_results, digits = 4))
} else {
  cat("**Results placeholder.** No K-group output is available.\n\n")
}

```

scenario	rejection_rate	mc_se	n_sim
null	0.1	0.0949	10
ordered	0.8	0.1265	10

Use of AI-assisted development

AI assistance was used during development to help locate and summarize published methods, draft documentation and simulation scaffolding, translate published formulas into candidate R code, and propose test cases. AI output was not treated as statistical validation. Candidate implementations were reviewed against the cited formulas. The executable checks below assess WC and PSH against independent R packages; these checks do not validate every rank test or competing-risk procedure. Package unit tests, vignette rendering, `roxygen2::roxygenise()`, and `devtools::check()` provide additional software validation. Completed simulation cells are distinguished from pending cells and from runs with incompatible generator metadata.

Discussion

The main contribution of `recurSurvTests` is not a claim that one estimator or test fits every recurrent-event question. Rather, it makes the choice visible: PSH/pooled KM for a renewal/IID gap interpretation, Wang–Chang for marginal gap-time survival with event-count weighting, Luo–Huang for the corresponding weighted-risk-set rank comparison, Zhao-style sensitivity analyses, and RCIF methods for recurrent competing risks. The reference checks

connect the new implementations to legacy software, while the simulation infrastructure makes remaining empirical questions reproducible rather than implicit.

Future work includes completing the configured Zhao and PSG simulations, resolving the observed reproduction discrepancies, further validating the implemented K-group rank-test extension, and comparing nonparametric tests with marginal and frailty regression models on planned sleep-bout applications.

Computational reproducibility

From the repository root, render with:

```
quarto render vignettes/jss-manuscript.qmd
```

This requires Quarto and R packages `here`, `knitr`, `rmarkdown`, `pkgload`, and `survival`. The manuscript loads the package source from this checkout with `here::i_am("vignettes/jss-manuscript.qmd")` anchoring the repository root and `pkgload::load_all()`, so a stale installed `recurSurvTests` is not used. `survrec` and `newTestSurvRec` are optional for their respective comparison rows. Rendering installs nothing, launches no simulation arrays, and disables chunk caching so newly copied result files are incorporated immediately. Seeds are specified in the executable examples and simulation drivers.

By default, Zhao input files are read from `data-raw/zhao-2020-table-simulations/`. To use another directory:

```
ZHAO_RESULTS_DIR=/path/to/completed/rds/files quarto render vignettes/jss-manuscript.qmd
```

The input must contain one RDS per task using the original 120-task partition. Missing files yield pending cells; duplicate tasks, incompatible settings, invalid summaries, or mixed variance conventions stop rendering with an error. An empty input directory renders the published benchmarks and placeholders. Long-running jobs must be submitted separately, using the documented shell scripts. For corrected log-logistic tasks only:

```
sbatch --array=4-120:4%20 data-raw/zhao-2020-table-simulations.sbatch
```

Archive any existing legacy log-logistic files outside the input directory before rerunning. The other distributions do not require regeneration for the log-logistic correction. Corrected generator metadata is checked per file; old outputs are never relabeled by the manuscript.

Each render exports the full Zhao comparison, the three publication-shaped tables, the reference-package checks, a standalone PNG and PDF comparison figure, input-file checksums, current source checksums, and R session information to `docs/jss-manuscript/` (override with `MANUSCRIPT_OUTPUT_DIR`). Input checksums identify the exact files used. Current source checksums document the rendering environment; they cannot recover the execution commit of legacy simulation files that did not record it. The benchmark transcription and its provenance note are included in the source manifest. The bibliography is maintained in `vignettes/references.bib` and rendered through standard Pandoc citations.

R version 4.4.0 (2024-04-24)

Platform: x86_64-apple-darwin20

Running under: macOS 26.6.2

Matrix products: default

BLAS: /Library/Frameworks/R.framework/Versions/4.4-x86_64/Resources/lib/libRblas.0.dylib

LAPACK: /Library/Frameworks/R.framework/Versions/4.4-x86_64/Resources/lib/libRlapack.dylib

locale:

[1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8

time zone: America/New_York

tzcode source: internal

attached base packages:

[1] stats graphics grDevices utils datasets methods base

342 other attached packages:

343 [1] recurSurvTests_0.0.1 testthat_3.3.2 colorout_1.3-0.2

344

345 loaded via a namespace (and not attached):

346 [1] newTestSurvRec_1.0.2 cli_3.6.6 knitr_1.51 rlang_1.2.0
347 [5] xfun_0.57 ote1_0.2.0 pkgload_1.5.2 jsonlite_2.0.0
348 [9] rprojroot_2.1.1 htmltools_0.5.9 survrec_1.2-5 tinytex_0.59
349 [13] pkgbuild_1.4.5 brio_1.1.5 rmarkdown_2.31 grid_4.4.0
350 [17] evaluate_1.0.5 fastmap_1.2.0 yaml_2.3.12 lifecycle_1.0.5
351 [21] compiler_4.4.0 here_1.0.2 rstudioapi_0.18.0 lattice_0.22-6
352 [25] digest_0.6.39 R6_2.6.1 splines_4.4.0 magrittr_2.0.5
353 [29] Matrix_1.7-2 tools_4.4.0 boot_1.3-31 survival_3.8-3
354 [33] desc_1.4.3

355 References

- 356 [1] P. K. Andersen and R. D. Gill. Cox’s regression model for counting processes: A large
357 sample study. *The Annals of Statistics*, 10(4):1100–1120, 1982. doi: 10.1214/aos/1176
358 345976.
- 359 [2] R. L. Prentice, B. J. Williams, and A. V. Peterson. On the regression analysis of
360 multivariate failure time data. *Biometrika*, 68(2):373–379, 1981. doi: 10.1093/biomet
361 /68.2.373.
- 362 [3] Terry M. Therneau. *coxme: Mixed Effects Cox Models*, 2024. URL [https://CRAN.R-](https://CRAN.R-project.org/package=coxme)
363 [project.org/package=coxme](https://CRAN.R-project.org/package=coxme). R package version 2.2-22.
- 364 [4] Virginie Rondeau, Yassin Mazroui, and Juan R. Gonzalez. frailtypack: An R package
365 for the analysis of correlated survival data with frailty models using penalized likelihood
366 estimation or parametrical estimation. *Journal of Statistical Software*, 47(4):1–28, 2012.
367 doi: 10.18637/jss.v047.i04.
- 368 [5] Juan R. Gonzalez, Edsel A. Peña, and Robert L. Strawderman. *survrec: Survival*

Analysis for Recurrent Event Data, 2018. URL <https://github.com/isglobal-brge/survrec>. R package version 1.2-5.

[6] Carlos Miguel Martinez Manrique. *newTestSurvRec: Statistical Tests to Compare Curves with Recurrent Events*, 2018. URL <https://CRAN.R-project.org/package=newTestSurvRec>. R package version 1.0.2.

[7] Mei-Cheng Wang and Shu-Hui Chang. Nonparametric estimation of a recurrent survival function. *Journal of the American Statistical Association*, 94(445):146–153, 1999. doi: 10.1080/01621459.1999.10473831.

[8] Xianghua Luo and Chiung-Yu Huang. Analysis of recurrent gap time data using the weighted risk-set method and the modified within-cluster resampling method. *Statistics in Medicine*, 30(4):301–311, 2011. doi: 10.1002/sim.4074.

[9] Edsel A. Peña, Robert L. Strawderman, and Myles Hollander. Nonparametric estimation with recurrent event data. *Journal of the American Statistical Association*, 96(456):1299–1315, 2001. doi: 10.1198/016214501753381887.

[10] Terry M. Therneau. *A Package for Survival Analysis in R*, 2024. URL <https://CRAN.R-project.org/package=survival>. R package version 3.8-3.

[11] Qiang Zhao, Bin Zhang, Michael P. LaValley, Joseph M. Massaro, Kathryn L. Lunetta, and Mark Chang. Extended rank tests for analyzing recurrent event data. *Statistics in Biopharmaceutical Research*, 12(1):90–98, 2020. doi: 10.1080/19466315.2019.1601596.

[12] Sisuma Mandakathingal Sivadasan and Paduthol Godan Sankaran. Nonparametric estimation of cumulative incidence functions of recurrent events. *Statistica*, 83(1):3–25, 2023. doi: 10.6092/issn.1973-2201/11727.

[13] M. S. Sisuma and P. G. Sankaran. Non-parametric test of recurrent cumulative incidence functions for competing risks models. *METRON*, 80(3):331–342, 2022. doi: 10.1007/s40300-022-00228-x.

[14] David Moriña and Albert Navarro. The R package survsim for the simulation of simple and complex survival data. *Journal of Statistical Software*, 59(2):1–20, 2014. doi: 10.18637/jss.v059.i02.